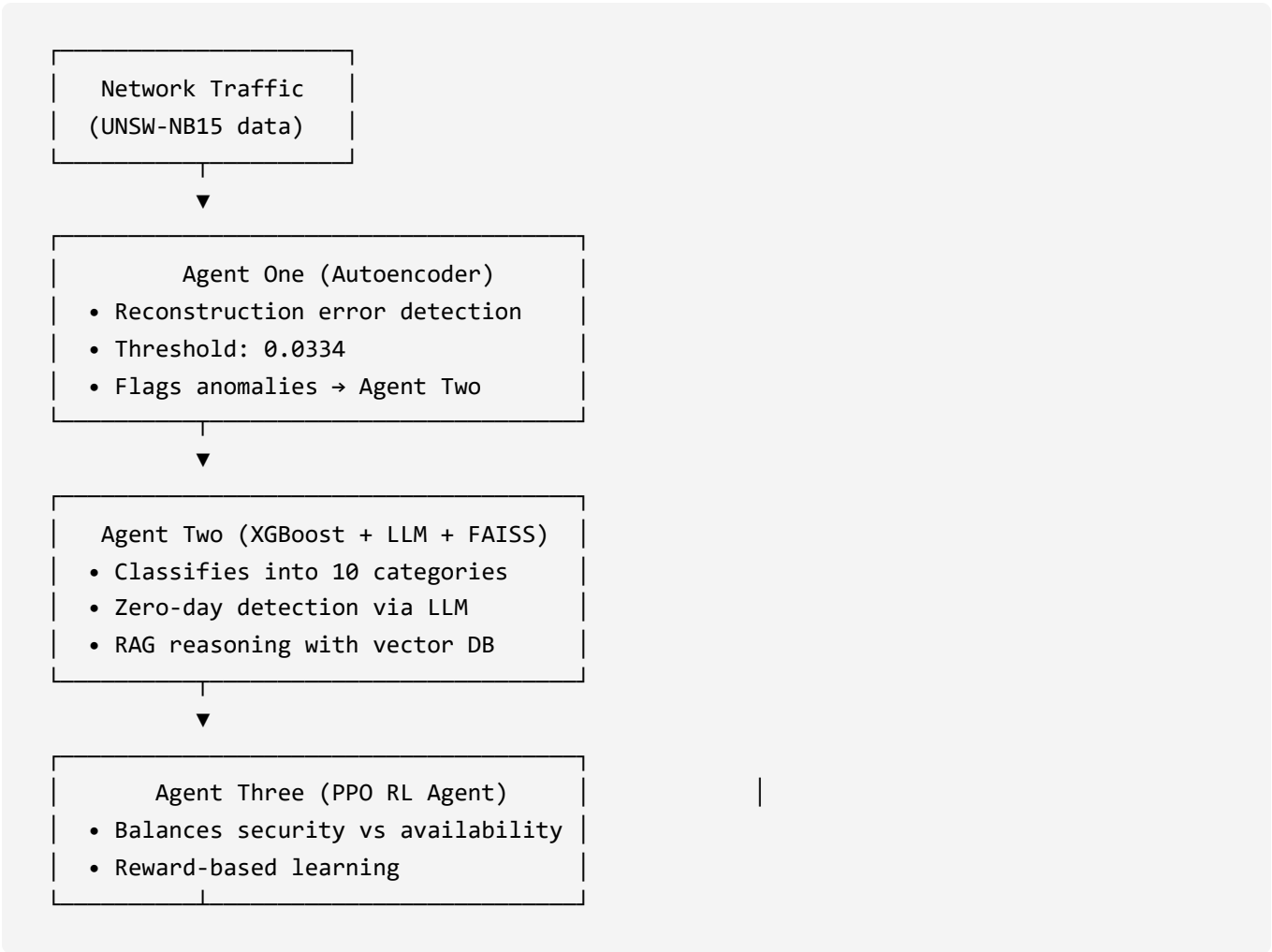


This is a **multi-agent intrusion detection system (IDS)** combining deep learning, gradient boosting, reinforcement learning, RAG (Retrieval-Augmented Generation), and federated learning for privacy-preserving network defense. The framework is architecturally ambitious and targets a real-world problem of significant importance.

A three-agent system for network intrusion detection using the **UNSW-NB15** dataset:

Agent	Role	Technology
Agent One	Anomaly Detection	PyTorch Autoencoder
Agent Two	Classification & Reasoning	XGBoost + LangChain/LLM
Agent Three	Mitigation Decision	PPO (Stable Baselines 3)

Architecture Flow



Architecture: Three-Agent Pipeline

```
Network Traffic → [Agent One] → [Agent Two] → [Agent Three] → Response
                  Autoencoder      XGBoost+RAG      Reinforcement
                  Anomaly          Classifier         Learning
                  Detection        + LLM              Mitigation
```

1. Data Pipeline

Components

- **DataLoader**: Loads UNSW-NB15 CSV files, handles dtypes
- **Preprocessor**: Encodes categorical features, scales numerical features
- **BatchGenerator**: Yields batches for training

2. Agent One: Anomaly Detection (Autoencoder)

- Trains an autoencoder on **normal traffic only**
- Normal traffic → low reconstruction error
- Attacks → high reconstruction error (anomaly)
- Threshold-based classification: `error > threshold → anomaly`

Why Keep Agent One?

Factor Analysis

- Computational efficiency Autoencoder is ~10x faster than XGBoost classification. At 100K flows/sec, filtering 50% with Agent One saves massive compute.
- Zero-day detection Autoencoder detects any anomaly, even novel attacks XGBoost wasn't trained on.
- Current state Agent One only missed 22 attacks (1.35%). That's excellent recall.

Note on FPR

1. The threshold was optimized for high recall ($\geq 98\%$)
2. To reduce FPR further would require sacrificing recall
3. In IDS, missing attacks (false negatives) is typically more dangerous than false alarms

To further reduce FPR while keeping high recall, consider:

- Training Agent Two to better filter false positives after Agent One
- Using ensemble methods
- Feature engineering to better separate normal/attack patterns

3. Agent Two: Classification & LLM Reasoning

1. **XGBoost Classifier:** Classifies threats into 10 categories:
 - Normal, Fuzzers, Analysis, Backdoor, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms
2. **Zero-Day Detection:** Low-confidence predictions trigger LLM reasoning
3. **RAG Pipeline:** FAISS vector DB retrieves similar attacks for context
4. **LLM Reasoning:** Generates analysis for unknown threats

Aspect	Details
Model	XGBoost Classifier (200 estimators, depth 8)
Purpose	Classify anomalies into attack categories
Categories	10 classes: Normal, Fuzzers, Analysis, Backdoor, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms
Zero-Day Detection	If confidence < 50%, flags as "Unknown/Zero-day"
RAG Integration	Vector DB + LLM for contextual reasoning on unknown threats

4. Agent Three: RL Mitigation Agent

- **Environment:** Custom Gymnasium environment (`NetworkDefenseEnv`)
- **State:** 14-dim vector (category one-hot + confidence + severity + zero_day + high_risk)

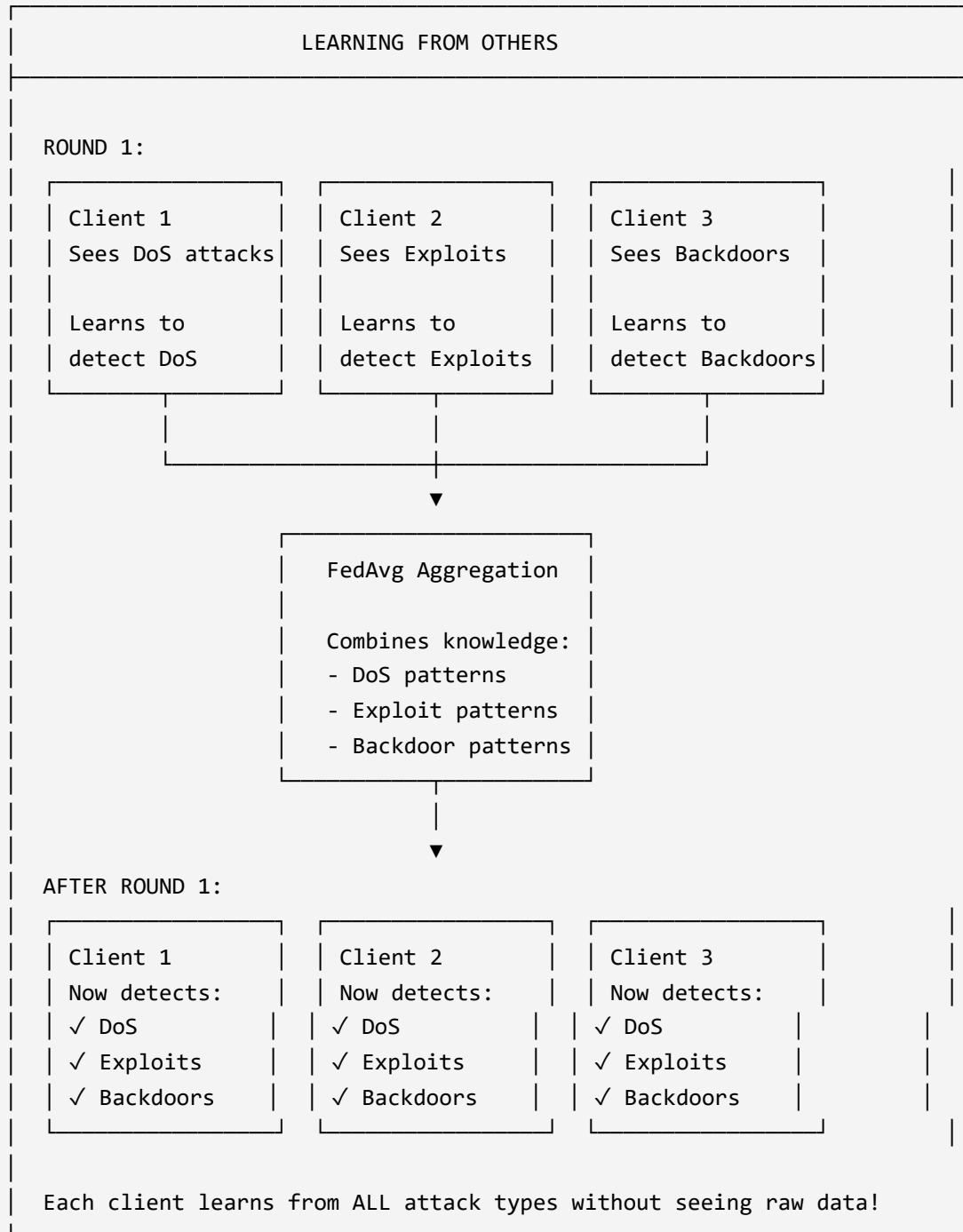
Aspect	Details
Model	PPO (Stable Baselines 3)
Purpose	Learn optimal mitigation actions
State	Threat category, severity, confidence, historical success
Reward	Balances security (blocking threats) vs. availability (not disrupting normal traffic)

5.Federated Learning

- **Privacy-Preserving:** Raw data never leaves local nodes; only model weights transmitted
- **FedAvg Aggregation:** Weighted averaging of Autoencoder + XGBoost models
- Flower Bagging for XG booster - In this architecture, the server treats each client's local training phase as a bootstrap sample. It collects the raw trees from all participating nodes and concatenates them together into a single, unified global model. If Client 1 trains 10 trees and Client 2 trains 10 trees, the server combines them into a 20-tree global forest.

- **SemanticThreatReport:** MITRE ATT&CK mapping, CVE references, severity levels
- **Graceful Fallbacks:** Handles missing agents, RAG failures, server disconnects
- Differential Privacy (DP) Implementation
- FL-RAG Bridge (Core Innovation)

How Learning From Others Works



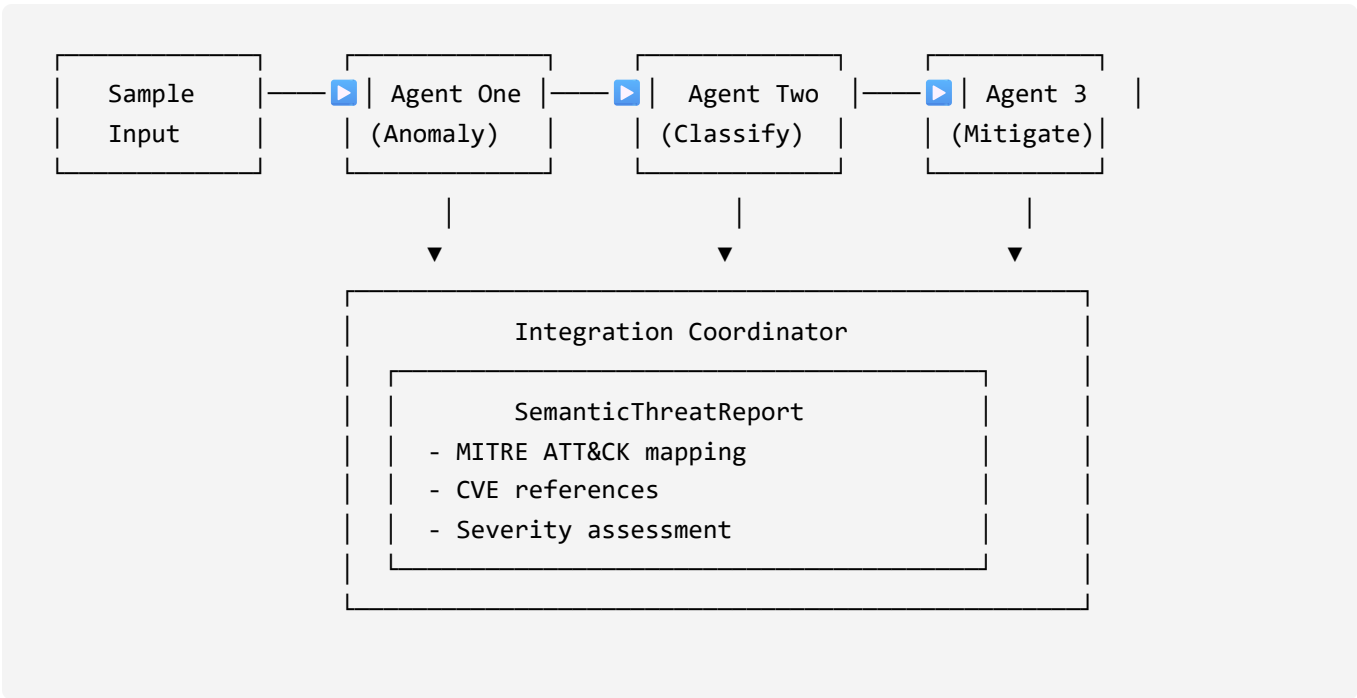
5.1 FL-RAG Bridge (Core Innovation)

In federated learning (FL), multiple distributed clients collaboratively train models by sharing **only model weights** (not raw data). When the global model improves, it learns new threat patterns from other organizations. However:

Traditional FL	FL + RAG Bridge
Model detects threat better	Model detects + explains why
Returns "DoS" classification	Returns context about CVE-2021-44228, MITRE T1498
Silent knowledge improvement	Triggers re-analysis when weights drift
Local explanations only	Global threat intelligence grounded explanations

1. **Interprets Model Weight Changes** → Determines what categories of threats the global model improved at detecting
2. **Triggers RAG Refresh** → Automatically refreshes knowledge retrieval when models change significantly
3. **Grounds Explanations in Global Intelligence** → Uses MITRE ATT&CK, CVE databases, and pattern libraries
4. **Tracks Explanation Quality Across Rounds** → Compares how explanations improve after federated updates

Pipeline Flow



MITRE ATT&CK Mapping

Attack Category	MITRE Technique	Tactic
DoS	T1498	Impact
Reconnaissance	T1595	Reconnaissance
Exploits	T1190	Initial Access
Backdoors	T1059	Execution
Shellcode	T1055	Privilege Escalation
Worms	T1080	Lateral Movement
Analysis	T1040	Collection
Fuzzers	T1499	Impact